Ex.No: Date:

Writing Wireshark Filter Expressions for Packet Capture

AIM:

To learn how to write and apply Wireshark filter expressions - both capture filters and display filters - so that only relevant packets are captured or shown during network analysis.

SOFTWARE REQIIRED:

- Computer with an active network interface (Ethernet or Wi-Fi).
- Wireshark (version 3.x or later) installed with WinPcap/Npcap (Windows)
- Administrator/root privileges (packet capture needs elevated rights).

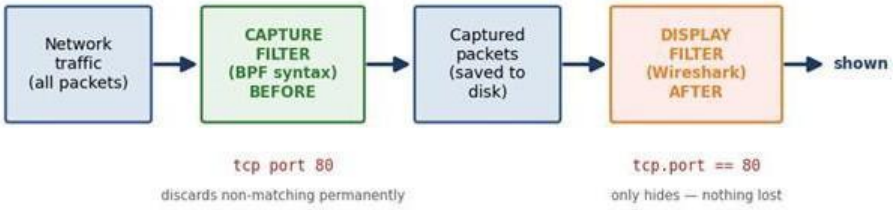
THEORY:

Wireshark is an open-source protocol analyser that captures live traffic from a network interface and lets you examine each packet field by field. Because a busy network produces thousands of packets per second, filters are essential to isolate the traffic you actually care about. Wireshark supports two distinct kinds of filters, and confusing them is the most common mistake made by beginners.

Capture filters vs. Display filters

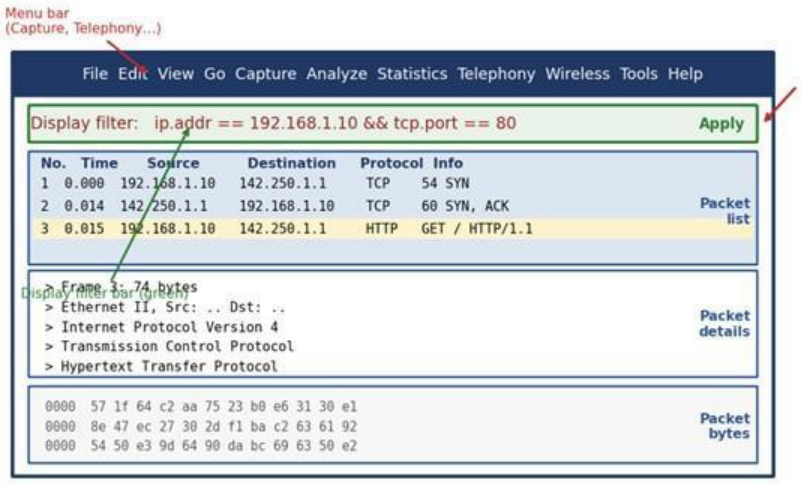
Aspect	**Capture Filter**	**Display Filter**
When applied	Before capture starts	After packets are captured
Syntax	BPF / libpcap syntax	Wireshark display-filter syntax
Effect	Discards non-matching packets permanently	Only hides packets; nothing is lost
Example	tcp port 80	tcp.port == 80
Where typed	Capture > Options dialog	Green bar below the toolbar

A **capture filter** reduces the volume of data written to disk and cannot be changed once capture has begun. A **display filter** is applied to already-captured data, can be changed at any time, and is far more expressive because it can reach into individual protocol fields.



Display-filter comparison and logical operators

Operator	Alias	Meaning	Example
==	eq	Equal to	ip.addr == 10.0.0.1
!=	ne	Not equal to	tcp.port != 22
>	gt	Greater than	frame.len > 1000
<	lt	Less than	tcp.window_size < 500
&&	and	Logical AND	ip.src==10.0.0.1 && tcp.port==80
II	or	Logical OR	dns II http
!	not	Logical NOT	!arp



PROCEDIRE:

1. Launch Wireshark. The welcome screen lists all available network interfaces along with a live sparkline of activity.
2. To use a **capture filter**, go to **Capture Options**. In the "Capture filter for selected interfaces" box, type an expression such as host 192.168.1.10. A green box means the syntax is valid; red means it is invalid.
3. Select the interface you want (e.g., Wi-Fi or Ethernet) and click Start. Only packets matching the capture filter are recorded.
4. Generate some traffic - open a web page, run a ping, or send an email - so there is data to observe.
5. To narrow what is shown, type a **display filter** into the green filter bar at the top (for example http.request) and press Enter.
6. Click any packet to inspect it: the middle pane shows the decoded protocol tree and the bottom pane shows the raw bytes in hex and ASCII.
7. Click the "X" in the filter bar to clear the display filter and see all captured packets again.
8. Stop the capture with the red square button and, if required, save the capture as a .pcapng file via File - Save As

Worked Examples of Filter Expressions

(a) Capture filters (typed in Capture > Options)

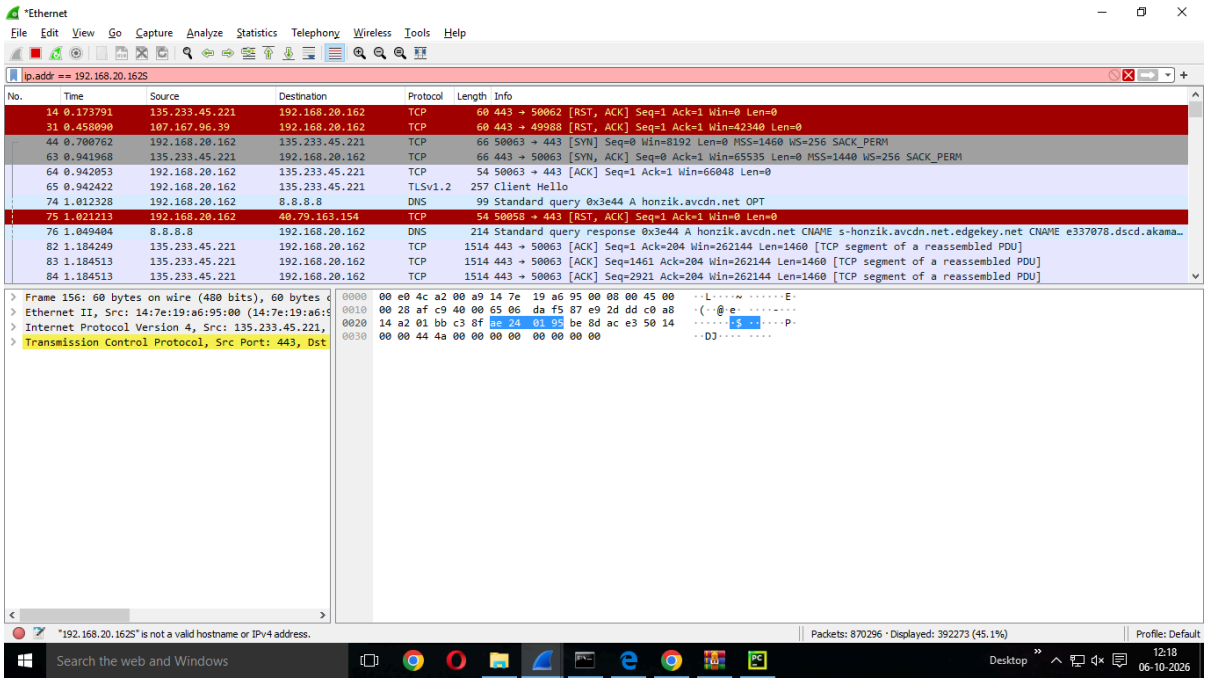
```
host 192.168.1.10          # all traffic to/from this host
net 192.168.1.0/24         # entire subnet
src host 192.168.1.10              # only packets FROM this
host dst port 53 # traffic going TO port 53 (DNS)
tcp port 443               # only HTTPS/TLS
traffic udp        # only UDP packets
not arp and not broadcast          # ignore ARP and
broadcasts tcp port 80 and host 192.168.1.10  # combined
condition
```

(b) Display filters (typed in the green filter bar)

```
ip.addr == 192.168.1.10                    # any packet with this IP
(src or dst) ip.src == 192.168.1.10                  # only where this IP
is the source tcp.port == 80       # HTTP over TCP
http      # only HTTP protocol packets
dns       # only DNS packets http.request.method == "GET"
# only HTTP GET requests
tcp.flags.syn == 1 && tcp.flags.ack == 0 # TCP connection starts (SYN)
tcp.analysis.retransmission        # retransmitted TCP segments
ip.addr == 192.168.1.10 && tcp.port == 443 # host AND port together
frame contains "password"        # packets whose bytes contain the text
```

SAMPLE OBSERVATION TABLE:

Filter Applied	Purpose	Result Observed
dns	Isolate name-resolution traffic	Only UDP/port-53 query & response pairs listed
tcp.flags.syn==1 && tcp.flags.ack==0	Find new connection attempts	One packet per handshake start
http.request	See outbound web requests	GET/POST lines with host & URI visible
ip.addr==8.8.8.8	Track one destination	All packets to/from 8.8.8.8 only



RESILT:

Thus the capture filters and display filters were written and applied successfully in Wireshark, and the difference between the two - syntax, timing, and effect - was verified through the observed output.

Ex.No: Date:

Analysis of RTP Packet Delay and Loss using Wireshark

AIM:

To capture Real-time Transport Protocol (RTP) traffic from a VoIP/streaming session and analyse packet delay (jitter), packet loss and out-of-order delivery using the telephony analysis tools built into Wireshark.

SOFTWARE REQIIRED:

- Computer with an active network interface (Ethernet or Wi-Fi).
- Wireshark (version 3.x or later) installed with WinPcap/Npcap (Windows)
- Administrator/root privileges (packet capture needs elevated rights).

THEORY:

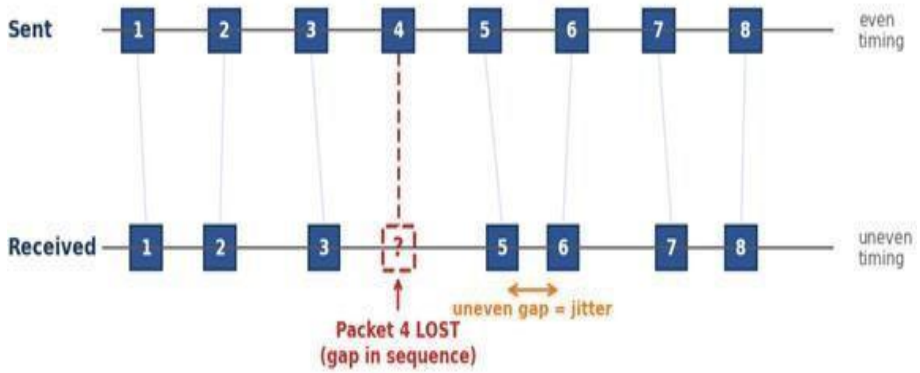
RTP (Real-time Transport Protocol) carries audio and video for VoIP and streaming applications. It runs over UDP because retransmitting late media is useless - a delayed voice sample is better dropped than replayed. Each RTP packet carries a sequence number and a timestamp, which together allow the receiver (and Wireshark) to detect the three quality problems that matter for real-time media:

- **Packet loss** - a gap in the RTP sequence numbers means one or more packets never arrived.
- **Jitter (delay variation)** - the variation in the inter-arrival time of packets. Even if average delay is fine, high jitter causes choppy audio because packets arrive unevenly.
- **Out-of-order I late packets** - packets arriving after the jitter buffer has already played their slot are effectively lost.

RTP is usually accompanied by RTCP (RTP Control Protocol), which periodically reports statistics such as cumulative loss and jitter between the endpoints. Wireshark understands both.

Key metrics Wireshark reports

Metric	Meaning	Acceptable value (voice)
Max / Mean Jitter	Variation in packet arrival time	< 30 ms mean is good
Lost packets (%)	Sequence-number gaps as a fraction	< 1 % is good; > 5 % is poor
Delta (ms)	Time since previous packet in the stream	~20 ms for G.711 at 50 pkt/s
Skew	Drift between expected & actual arrival	Near 0 is ideal



RTP Stream Jitter and Packet loss

PROCEDIRE:

1. Start Wireshark and begin capturing on the interface that will carry the call.
2. Place a VoIP call between the two endpoints (or open an existing VoIP .pcap file via File Open).
3. Let the call run for at least 20-30 seconds so enough media flows, then stop the capture.
4. If the media is not automatically recognised as RTP, right-click a UDP packet from the media stream, choose **Decode As.**, and set the protocol to RTP.
5. Open **Telephony --- RTP RTP Streams**. Wireshark lists every RTP stream with source/destination addresses, payload codec, packet count, lost packets and maximum jitter.
6. Select the stream of interest and click Analyze. The per-packet analysis window opens.
7. Study the Analyze window columns - Sequence number, Delta (ms), Jitter (ms), Skew and Status. Any row marked with a warning (e.g., "Wrong sequence number" or "Incorrect timestamp") indicates loss or reordering.
8. Read off the summary line at the bottom: total packets, **lost packets and percentage**, **max jitter** and **mean jitter**.
9. For a live quality feel, click **Play Streams** to hear the reconstructed audio, or use **Telephony VoIP Calls** for a call-level view and a flow graph.
10. Optionally view control statistics with the display filter rtcp, which shows the loss and jitter each endpoint reports.



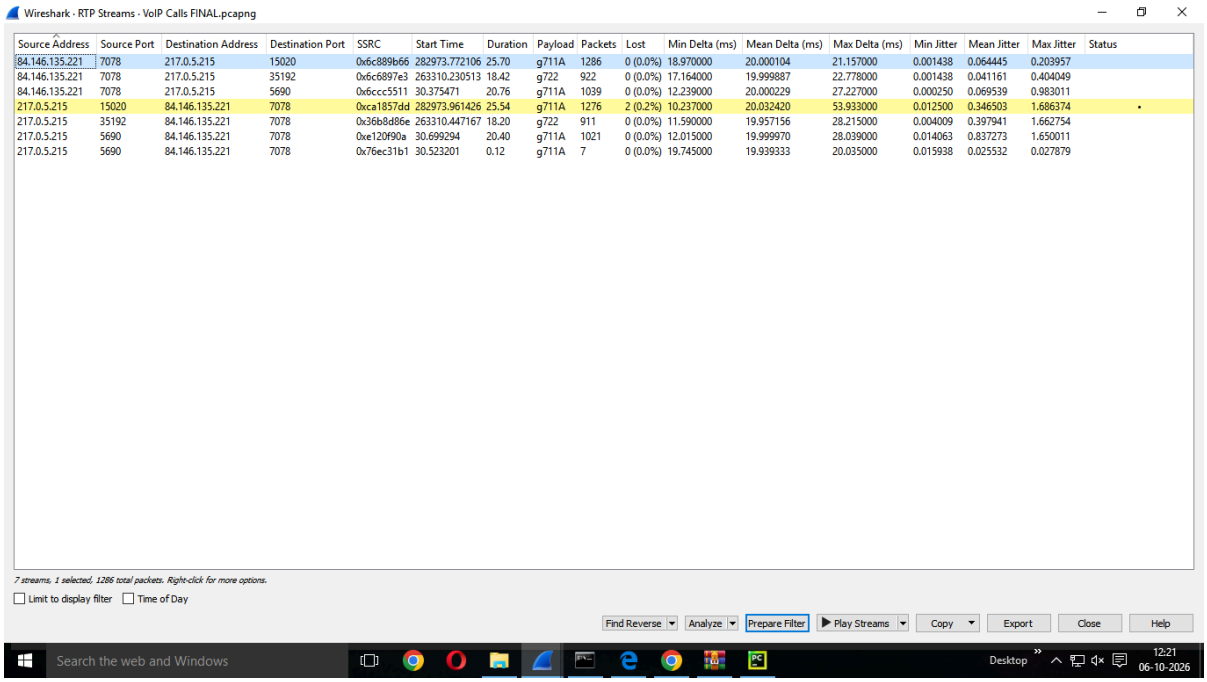
Telephony ->RTP->RTP Streams Window

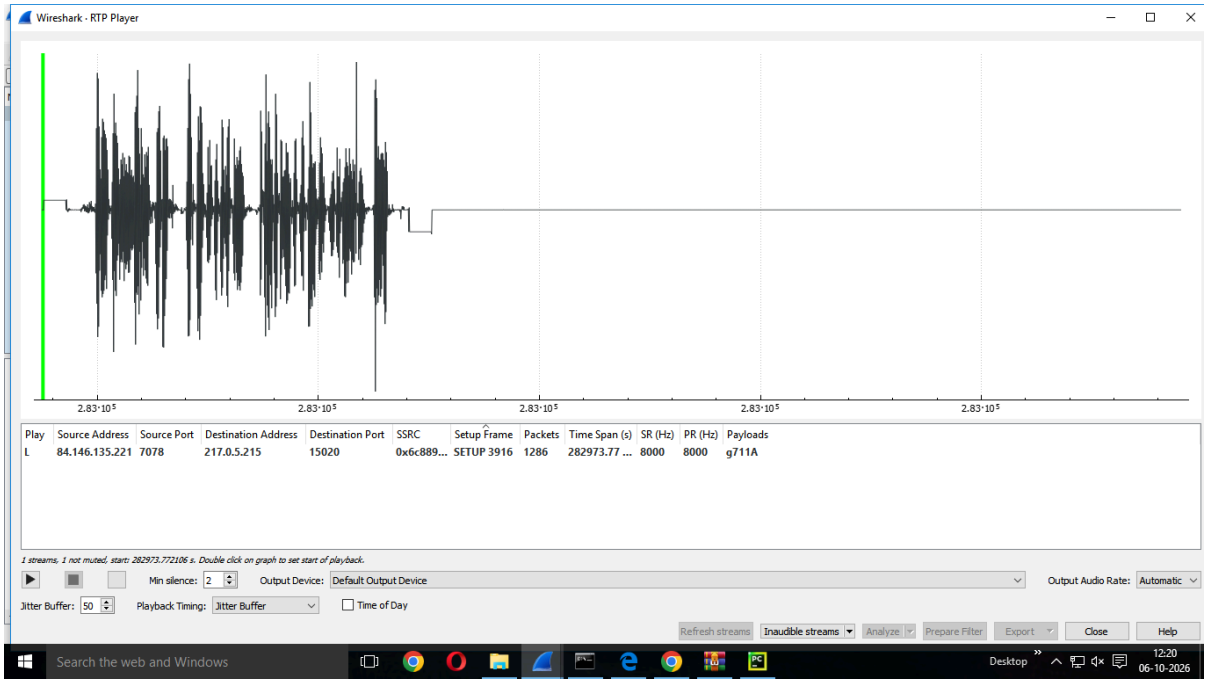
Useful Display Filters

SAMPLE OBSERVATION TABLE:

Fill in the values read from the RTP Streams and Analyze windows for your own capture:

Parameter	Stream A - B	Stream B - A
Codec / Payload	G.711 (PCMU)	G.711 (PCMU)
Total packets	1500	1498
Lost packets (count / %)	6 / 0.40 %	2 / 0.13 %
Max jitter (ms)	18.4	12.1
Mean jitter (ms)	6.2	4.9
Max delta (ms)	42.7	31.5
Quality assessment	Good	Good





RESILT:

Thus an RTP stream was captured and analysed in Wireshark. Packet delay (jitter), packet loss and out-of-order delivery were measured using the RTP Stream

Ex.No: Date:

Ise Wireshark to Inspect Packets on Your Network

AIM:

To capture live traffic from your own network interface and inspect the packets - examining protocol layers, following a TCP conversation, and reading traffic statistics - in order to understand how data is exchanged on the network.

SOFTWARE REQIIRED:

- A computer connected to a live network (Ethernet or Wi-Fi).
- Wireshark installed with Npcap/libpcap and administrator privileges.
- A web browser or ping utility to generate observable traffic.

THEORY:

Every action on a network - loading a page, resolving a domain, sending a message - is broken into packets that travel between hosts. Wireshark places the network card into a mode where it can read these packets and decode them according to the TCP/IP model. Understanding a single packet means understanding the layered structure wrapped around the data:

Layer	**Protocol example**	**What Wireshark shows**
Physical/Data-Link	Ethernet II	Source & destination MAC addresses
Network	IPv4 / IPv6	Source & destination IP addresses, TTL
Transport	TCP / UDP	Port numbers, sequence & ack numbers, flags
Application	HTTP, DNS, TLS	Requests, responses, queries, payload

Wireshark's main window has three panes: the packet list (one line per packet), the packet details tree (the decoded layers of the selected packet), and the packet bytes pane (the same data in raw hex/ASCII).

lc:tl

192.1⬜8.l.10

SYN

SYN, ACK

ACK

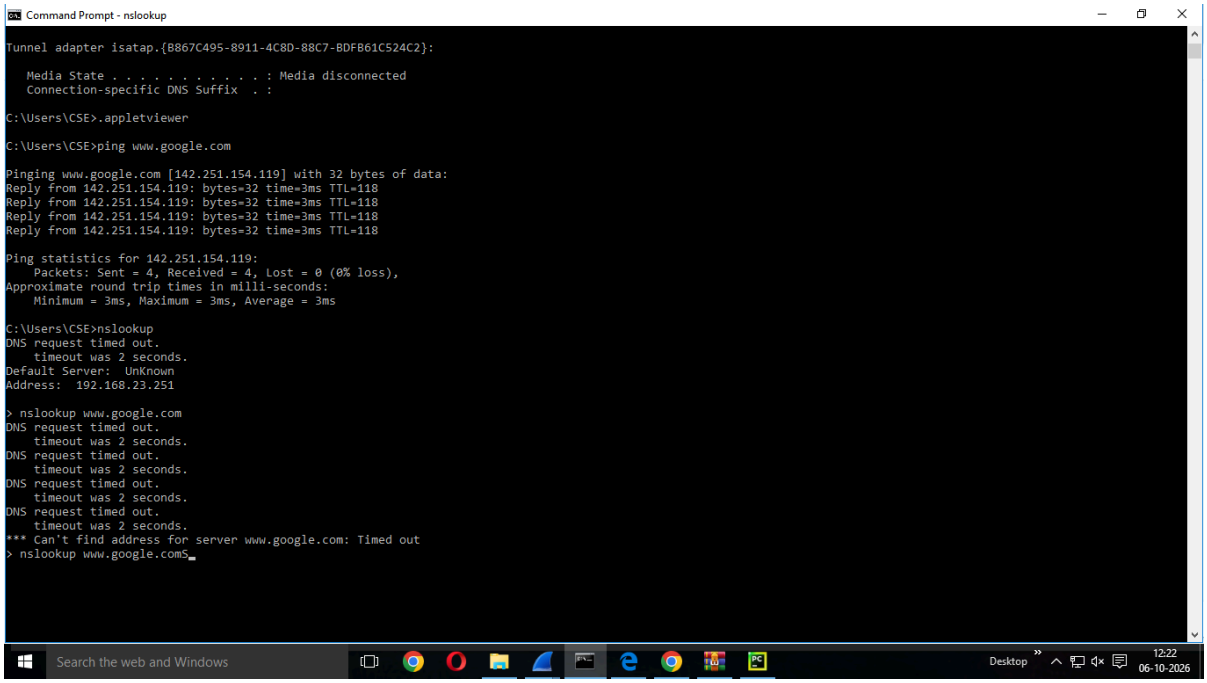
HTTP GET/ ...

HTTP 200 OK + data

ls:erl

142.2⬜0.l.l

lhtee.way h.,ndshakt + r⬜st1respot1st tHS$1H1'1b/ed mto **one***tHdablt* **sttea**



PROCEDIRE:

1. Open Wireshark. On the welcome screen, double-click the active interface (the one whose sparkline is moving) to begin capturing.
2. Generate traffic: open a website, or from a terminal run a ping so you have known packets to find.
3. ping www.google.com
   nslookup www.google.com
4. Watch the packet list fill in real time. Stop the capture (red square) after a few seconds so the list stops moving and is easier to read.
5. Apply a display filter to focus, e.g. type dns to see the name lookups, or icmp to see your ping echo requests and replies.
6. Click one packet and expand the details tree in the middle pane. Read down through Ethernet **&#65533;** IP **&#65533;** TCP/UDP **&#65533;** application layers and note the addresses and ports.
7. Right-click a TCP packet and choose **Follow &#65533; TCP Stream** to reassemble an entire conversation (for example the full HTTP request and response) into a single readable window.
8. Open **Statistics &#65533; Protocol Hierarchy** to see the percentage of traffic by protocol - a quick profile of what your machine is doing.
9. Open **Statistics &#65533; Conversations** to list every pair of hosts talking, with byte and packet counts, and **Statistics &#65533; Endpoints** to see the busiest addresses.
10. Optionally open **Statistics &#65533; I/O Graph** to plot traffic volume over time.
11. Save the capture via File &#65533; Save As if you need to submit it or analyse it later.

SAMPLE OBSERVATION TABLE:

Packet No.	Source IP	Destination IP	Protocol	Info
1	192.168.1.5	192.168.1.1	DNS	Standard query A www.google.com
2	192.168.1.1	192.168.1.5	DNS	Response A 142.250.x.x
3	192.168.1.5	142.250.x.x	TCP	SYN connection request
4	142.250.x.x	192.168.1.5	TCP	SYN, ACK accepted
5	192.168.1.5	142.250.x.x	ICMP	Echo (ping) request

RESILT:

Thus, the live packets were captured from the network interface and inspected in Wireshark.